Quest 5: The Cartograffity

Quest 4 Quest 6

Part I

Story section

Once the tournament results had finally been calculated, the elves officially declared Legolink Pointereye their greatest warrior. As promised, he joined our expedition. Before we set off, the Grand Council handed us several maps to guide us safely to Nulldor.

With an elven warrior leading the way, we expected a smooth journey. During our very first stop, however, Frodoff picked up one of the maps. He turned it one way, then the other, held it upside down, frowned, and started the whole process again.

Gimlock watched this strange performance for a moment before suggesting that the elves had probably made another perfectly sensible invention. Perhaps the missing details appeared only to people who already knew where they were going.

It turned out that neither explanation was correct. The route was perfectly visible, and the destination had been marked just as clearly. The problem was that nothing else was on the map. There were only lighter and darker patches, like the static on an old crystal ball that had lost its signal.

Gimlock stared at the map for a while before handing it back. "At least it's consistently elven." Since we would all need to use these maps on the journey to Nulldor, we agreed that everyone should learn how to read them. Legolink offered to explain everything from the beginning.

Legolink explained that the lighter and darker patches were not random at all. To make the maps easier to read, we could represent every lighter cell as `0` and every darker cell as `1` (your notes). Every map had an odd number of rows and an odd number of columns. This meant that it always had exactly one central cell.

To reconstruct the land, begin by drawing a single land cell, represented by `#`, in the exact centre of the map. From that point, new pieces of land are gradually added around the land that has already been discovered. Consider every cell that touches the current land horizontally, vertically or diagonally.

A cell becomes land if at least `5` of its eight neighbouring cells are either already marked as land `#`, or dark cells `1` on the original map. Whenever a cell meets this condition, replace it with `#`. It becomes part of the known land and may allow more cells around it to be attached. Continue until no more land cells can be added.

Example based on the following notes:

```
01010010010000110100
11110100100000111000
10111101100010101001 1
00100010101000010000 0
00011010011001100 1100
00000111011111100000 0
10110001010101111000 0
00110110000000101001 1
00100100111101000100 1
10001101111001101010 1
00110010111001001000 0
```

The first step is to mark the central cell of the map as land `#`. The diagrams below highlight all cells adjacent to the known land, including those touching only at a corner, as these are the only cells that can be added to the land next.

Everybody Codes is possible thanks to:

Jane Street

Jane Street is a quantitative trading firm with offices worldwide.
We hire smart, humble people who love to solve problems, build systems, and test theories. Will our next great idea come from you?

Find out more

```
01010010010000101101000
11110100100000111000000
10111101100010101001100
00100010101000010000000
00011010011001100100000
00000111011111000000000
10110001010101110000000
00110110000000101001100
00100100111101000100100
10001101111001101010100
00110010111001001000000
```

After repeatedly attaching every possible cell, the reconstructed land looks like this:

```
01010010010000101101000  01010010010000101101000  01010010010000101101000  01010010010000101101000
11110100100000111000000  11110100100000111000000  11110100100000111000000  11110100100000111000000
10111101100010101001100  10111101100010101001100  10111101100#00101010011  10111101##00101010011
00100010101000010000000  00100010101#10000100000  00100010101#1#0#0100000  0010001########100000
00011010011001100100000  00011010011####11001100  0001101#######1001100  000110#########001100
00000111011111000000000  00000111#####11000000000  00000111######1000000  000001#########000000
10110001011#10111100000  10110001011#10111100000  10110001######1110000  10110008########110000
00110110000000101001100  00110110011##0001010011  00110110####001010011  0011011#######1010011
00100100111101000100100  00100100111101000100100  00100100###1010001001  0010010####1010001001
10001101111001101010100  10001101111001101010100  10001101111001101010100  10001101###0011010101
00110010111001001000000  00110010111001001000000  00110010111001001000000  00110010111001001000000

01010010010000101101000  01010010010000101101000  01010010010000101101000  01#1001001000011010000
11110100100000111000000  11110100100000111000000  11110100100000111000000  11#######0000####0000
10111101100010101001100  10111101100010101001100  10111101100010101001100  10111101##00101010011
00100010101000010000000  00100010101000010000000  00100010101000010000000  0010001########100000
00011010011001100100000  00011010011001100100000  00011010011001100100000  000110#########001100
00000111011111000000000  00000111011111000000000  00000111011111000000000  000001#########000000
10110001011#10111100000  10110001011#10111100000  10110001011#10111100000  10110008########110000
00110110000000101001100  00110110000000101001100  00110110000000101001100  0011011#######1010011
00100100111101000100100  00100100111101000100100  00100100111101000100100  0010010####1010001001
10001101111001101010100  10001101111001101010100  10001101111001101010100  10001101###0011010101
00110010111001001000000  00110010111001001000000  00110010111001001000000  00110010111001001000000

01010010010000101101000  01010010010000101101000  01010010010000101101000  01#1001001000011010000
11110100100000111000000  11110100100000111000000  11110100100000111000000  11#######0000####0000
10111101100010101001100  10111101100010101001100  10111101100010101001100  10111101##00101010011
00100010101000010000000  00100010101000010000000  00100010101000010000000  0010001########100000
00011010011001100100000  00011010011001100100000  00011010011001100100000  000110#########001100
00000111011111000000000  00000111011111000000000  00000111011111000000000  000001#########000000
10110001011#10111100000  10110001011#10111100000  10110001011#10111100000  10110008########110000
00110110000000101001100  00110110000000101001100  00110110000000101001100  0011011#######1010011
00100100111101000100100  00100100111101000100100  00100100111101000100100  0010010####1010001001
10001101111001101010100  10001101111001101010100  10001101111001101010100  10001101###0011010101
00110010111001001000000  00110010111001001000000  00110010111001001000000  00110010111001001000000

01#1001001000011010000  01#1001001000011010000  ..#................
1########0000####0000  1########0000####0000  .########....####....
1#########00#####0011  1#########00#####0011  .#########..#####....
001###############000  00###############000  ..###############...
000#############1100  000###############100  ...###############...
000###############000  000###############000  ...###############...
101###############0000  101###############0000  ...###############....
00###############0011  00###############0011  ..###############....
001###############001  00###############001  ..###############...
10################101  10################101  ..###############...
0011001###10010010000  0011001###10010010000  .......###..........
```

The final diagram shows only land cells as `#` and water cells as `.` for better readability.
The reconstructed land contains `137` cells.

How many cells on the map are occupied by land?

Your notes for this part:

Copy Open Download

Part I solved with answer: 1230

Check your progress

Part II

Gimlock was still not impressed. He pointed out that such a map could not even tell the difference between solid land and a sandy beach. Legolink calmly picked up another map (your notes) and assured us that it could.

First, reconstruct the land exactly as in the previous part. Once the entire land surface has been found, the sandy coastline can be determined.

Every land cell that is adjacent to at least one water cell, including diagonal neighbours, immediately becomes sand `=`. Always treat the map as if it were surrounded by water on all sides, so every cell on the edge of the map is considered adjacent to water. The remaining sand is then found in much the same way as the land was in the previous part. A land cell becomes sand if at least `5` of its eight neighbouring cells are already marked as sand `=`. Continue until no more cells can become sand.

Example based on the following notes:

```
01010010010000011010000
11110100100000111000000
10111101100010101001100
00100010101000001000000
00011010011001110011000
00000111011111100000000
10110001010101111000000
00110110000000101001100
00100100111101000100100
10001101111001101010100
00110010111001001000000
```

This is exactly the same example as in the previous part, so we already know what the reconstructed land looks like.

```
..#................
.########....###....
.#########..#####....
..#################...
...################...
...################...
...###############....
..################....
..#################...
..#################...
.......###...........
```

First, every land cell adjacent to water becomes sand `=`. This includes diagonal neighbours. The map is also treated as if it were surrounded by water on all sides, so every land cell on its edge becomes sand as well.

```
..=................
.========....====....
.==#####==..==##=....
..=#####====###==...
...=#############=...
...=############==...
...=############=....
..=#############=....
..=##############==...
..======#==========...
.......===...........
```

The remaining sand is found in much the same way as the land in the previous part. During each step, every land cell with at least `5` neighbouring sand cells also becomes sand.

```
..=................   ..=................   ..=................
.========....====....   .========....====....   .========....====....
.===####==..==#==....   .====###==..=====....   .====###==..=====....
..=#####====###==...   ..==#####====###==...   ..==#####====##===...
...=#############==...   ...=#############==...   ...=#############==...
...=#############==...   ...=#############==...   ...=#############==...
...=############=....   ...=############=....   ...=############=....
..=#############=....   ..==#############=....   ..==#############=....
..=##############===...   ..===##############===...   ..===##############===...
..=================...   ..=================...   ..=================...
.......===...........   .......===...........   .......===...........
```

After the third growth step, no more land cells have at least five sand neighbours, so the process stops.
The reconstructed beach contains `69` sand cells.

How many cells on the map are occupied by sand?

Your notes for this part:

Copy Open Download

Part II solved with answer: 1387

Check your progress

Part III

Gimlock stared at the latest map in complete disbelief. "Don't tell me these maps can even show trees. What next, potato stamps?" Legolink quietly opened a small wooden chest filled with... potato stamps. Gimlock blinked. Without a word, Legolink picked up a potato stamp carved into the shape of a tree, then took out another map (your notes) to demonstrate the procedure.

First, reconstruct the land and sand exactly as before. Once both have been determined, the remaining land can be decorated with trees. A tree can be added at X whenever the following 5 by 5 pattern fits:

```
.+++.
+++++
++X++
+++++
.+++.
```

For every land cell, treat that cell as `X` and examine the area around it:

Dots `.` represent cells that are irrelevant. Every `+` marks a cell that must contain either land `#` or a tree `@`. If all cells marked with `+` meet this condition, the land cell at `X` becomes a tree `@`. Check every land cell in this way.

Once all possible trees have been added with the stamp, the forest grows for exactly two more steps. During each step, a land cell becomes a tree if:

- at least `4` of its eight neighbouring cells are already trees, and
- none of its eight neighbouring cells is sand.

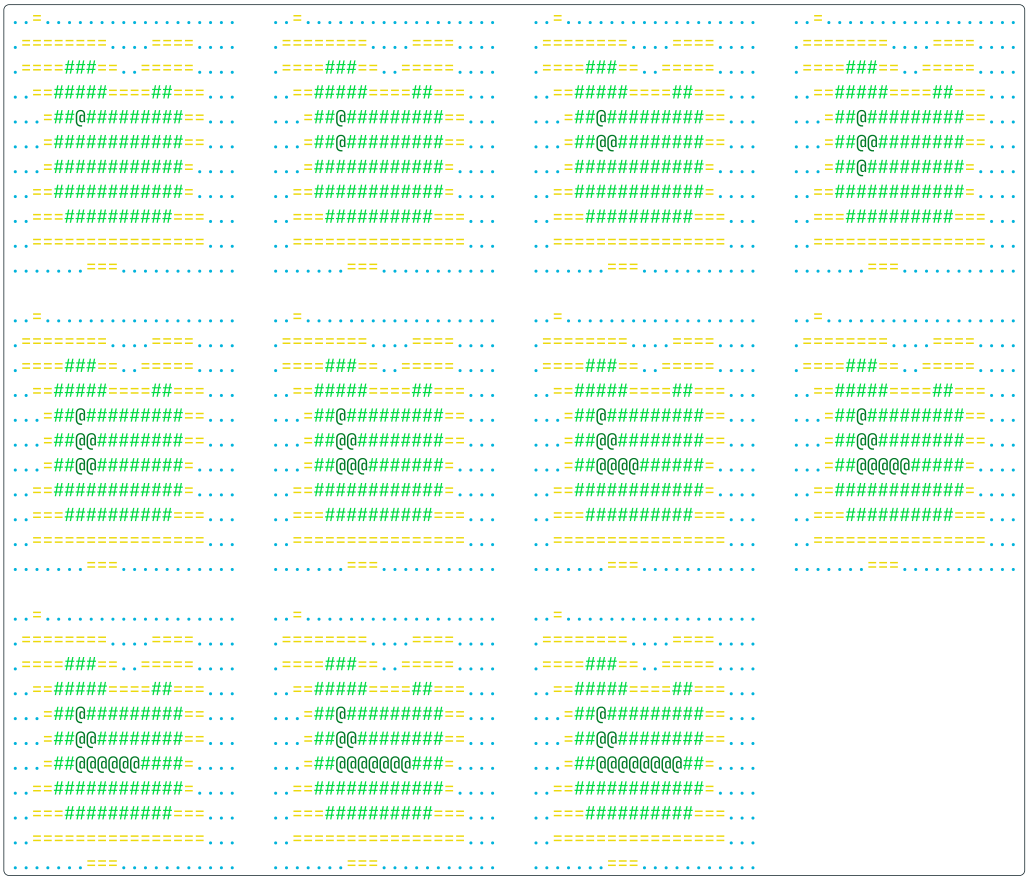
Each of these two steps is applied simultaneously across the entire map. This means that trees added during a step may only affect the following step.

Example based on the following notes:

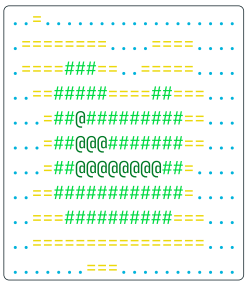
```
01010010010000011010000
11110100100000011100000
10111101100010101001 1
00100010101000010000 0
00011010011001100 1100
00000111011111100000 0
10110001010101111000 0
00110110000000101001 1
00100100111101000100 1
10001101111001101010 1
00110010111001001000 0
```

This is the same map as in the previous parts, so the land and sand have already been determined. Legolink now checks the remaining land cells from top to bottom and from left to right. Whenever all cells marked with `+` contain land or trees, the cell at `X` becomes a tree `@`. Trees added earlier may therefore make later stamp positions valid.

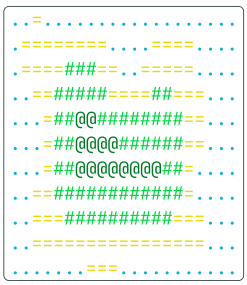
The maps below show every successful use of the stamp. The two forest-growth steps are shown separately afterwards.



During the first growth step, 1 additional land cell becomes tree:



During the second growth step, 2 additional land cells become trees:



The final map contains 14 tree cells.

How many cells on the map are occupied by trees?

Your notes for this part:

Copy Open Download

Part III solved with answer: 9334

Check your progress

Puzzle solved! Don't stop now!

Post your solution, compare ideas, and help others grow on Reddit or join us for a chat on Discord

Bonus for Supporters!

This feature is available only to Supporters as a small "thank you" for supporting this event. Click the link below to get all 100 versions of the input notes prepared for this Quest, along with the answers! Please don't host it anywhere; especially please don't add it to your solutions repo. Thank you for being awesome! ❤

Download

© 2024-2026 Everybody Codes. All right reserved.
Terms of Use, Privacy Policy, Cookies and Tracking Policy
By using this website, you agree to these terms.